Software Requirements Engineering

Lecture 8: Software Requirements Specification II

Review

- Characteristics of requirement statements: Complete, Correct, Feasible, Necessary, Prioritized, Unambiguous, Verifiable
- Characteristics of requirements collections: Complete, Consistent, Modifiable, Traceable

Outline

1. Software requirement specification

Writing style

- Adjust your writing style to put the punch line—the statement of need or functionality—first, followed by supporting details (rationale, origin, priority, and other requirement attributes).
- Include tables, structured lists, diagrams, and other visual elements.
- Don’t use multiple terms for the same concept just to achieve variety.
- Being easy to read and understand is an essential element of well-written requirements.

Writing style – tips

- Clarity and conciseness
  - Write requirements in complete sentences using proper grammar, spelling, and punctuation.
  - Keep sentences and paragraphs short and direct.
  - Define specialized terms in a glossary.
- The keyword “shall”
  - “Shall” statements clearly indicate the desired functionality
- Active voice
  - Write in the active voice to make it clear what entity is taking the action described.
- Individual requirements
  - Avoid writing long narrative paragraphs that contain multiple requirements.

Writing style – Level of detail

- Requirements need to be specified at a level of precision that provides developers and testers with just enough information to properly implement them.

You should include more detail when:

- The work is being done for an external client.
- Development or testing will be outsourced.
- Project team members are geographically dispersed.
- System testing will be based on requirements.
- Accurate estimates are needed.
- Requirements traceability is needed.

It's safe to include less detail when:

- The work is being done internally for your company.
- Customers are extensively involved.
- Developers have considerable domain experience.
- Precedents are available, as when a previous application is being replaced.
- A package solution will be used

Writing style – Avoiding ambiguity

- Requirements quality is in the eye of the reader, not the author.
- Peer reviews are the best way to find places where the requirements aren't clearly understood by all the intended audiences.
- Common sources of requirements ambiguity:
- Fuzzy words: Use terms consistently and as defined in the glossary.
- The A/B construct: It's better to decide exactly what you intend to say and choose the right words.
- Boundary values: Many ambiguities occur at the boundaries of numerical ranges in both requirements and business rules.
- Negative requirements: Try to rephrase negative requirements into a positive sense that clearly describes the restricting behavior.

Sample requirements

Example 1 The Background Task Manager shall provide status messages at regular intervals not less than every 60 seconds.

- What are the status messages? Under what conditions and in what fashion are they provided to the user? If displayed on the screen, how long do they remain visible? Is it okay if they just flash up for half a second?

1. The Background Task Manager (BTM) shall display status messages in a designated area of the user interface.

1.1. The BTM shall update the messages every 60 plus or minus 5 seconds after background task processing begins.

1.2. The messages shall remain visible continuously during background processing.

1.3. The BTM shall display the percent of the background task that is completed.

1.4. The BTM shall display a “Done” message when the background task is completed.

1.5. The BTM shall display a message if the background task has stalled.

Sample requirements

Example 2 Corporate project charge numbers should be validated online against the master corporate charge number list, if possible.

- The phrase “if possible” is ambiguous. Does it mean “if it’s technically feasible” (a question for the developer) or “if the master charge number list can be accessed at run time”?

Sample requirements

Example 3 The system must check for inconsistencies in account data between the Active Account Log and the Account Manager archive. The logic that is used to generate these comparisons should be based on the logic in the existing consistency checker tool. In other words, the new code does not need to be developed from scratch. The developers should utilize the current consistency checker code as the foundation. However, additional logic must be added to identify which database is the authoritative source. The new functionality will include writing data to holding tables to indicate how/where to resolve inconsistencies. Additionally, the code should also check for exception scenarios against the security tools database. Automated email alerts should be sent to the Security Compliance Team whenever discrepancies are found.

Ambiguous terms	Ways to improve them
acceptable, adequate	Define what constitutes acceptability and how the system can judge this.
and/or	Specify whether you mean "and," "or," or "any combination of" so the reader doesn't have to guess.
as much as practicable	Don't leave it up to the developers to determine what's practicable. Make it a TBD and set a date to find out.
at least, at a minimum, not more than, not to exceed	Specify the minimum and maximum acceptable values.
best, greatest, most	State what level of achievement is desired and the minimum acceptable level of achievement.
between, from X to Y	Define whether the end points are included in the range.
depends on	Describe the nature of the dependency. Does another system provide input to this system, must other software be installed before your software can run, or does your system rely on another to perform some calculations or provide other services?
efficient	Define how efficiently the system uses resources, how quickly it performs specific operations, or how quickly users can perform certain tasks with the system.
fast, quick, rapid	Specify the minimum acceptable time in which the system performs some action.
flexible, versatile	Describe the ways in which the system must be able to adapt to changing operating conditions, platforms, or business needs.
i.e., e.g.	Many people are unclear about which of these means "that is" (i.e., meaning that the full list of items follows) and which means "for example" (e.g., meaning that just some examples follow). Use words in your native language, not confusing Latin abbreviations.
improved, better, faster, superior, higher quality	Quantify how much better or faster constitutes adequate improvement in a specific functional area or quality aspect.

including, including but not limited to, and so on, etc., such as, for instance	List all possible values or functions, not just examples, or refer the reader to the location of the full list. Otherwise, different readers might have different interpretations of what the whole set of items being referred to contains or where the list stops.
in most cases, generally, usually, almost always	Clarify when the stated conditions or scenarios do not apply and what happens then. Describe how either the user or the system can distinguish one case from the other.
match, equals, agree, the same	Define whether a text comparison is case sensitive and whether it means the phrase "contains," "starts with," or is "exact." For real numbers, specify the degree of precision in the comparison.
maximize, minimize, optimize	State the maximum and minimum acceptable values of some parameter.
normally, ideally	Identify abnormal or non-ideal conditions and describe how the system should behave in those situations.
optionally	Clarify whether this means a developer choice, a system choice, or a user choice.
probably, ought to, should	Will it or won't it?
reasonable, when necessary, where appropriate, if possible, as applicable	Explain how either the developer or the user can make this judgment.
robust	Define how the system is to handle exceptions and respond to unexpected operating conditions.

Writing style – Avoiding ambiguity

Ambiguous terms	Ways to improve them
seamless, transparent, graceful	What does "seamless" or "graceful" mean to the user? Translate the user's expectations into specific observable product characteristics.
several, some, many, few, multiple, numerous	State how many, or provide the minimum and maximum bounds of a range.
shouldn't, won't	Try to state requirements as positives, describing what the system will do.
state-of-the-art	Define what this phrase means to the stakeholder.
sufficient	Specify how much of something constitutes sufficiency.
support, enable	Define exactly what functions the system will perform that constitute "supporting" some capability.
user-friendly, simple, easy	Describe system characteristics that will satisfy the customer's usage needs and usability expectations.

Software Requirements Specification

- The software requirements document (also called software requirements specification or SRS) is an official statement of what the system developers should implement.
- The SRS should include both the user requirements for a system and a detailed specification of the system requirements.
- The SRS has a diverse set of users and they need different levels of details.
  - Software development teams need to know what to build.
  - Training personnel use the SRS and user documentation to develop educational materials.
  - Testers use it to develop requirements-based tests, test plans, and test procedures
  - …

The Structure of a Requirements Document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The Structure of a Requirements Document

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

An example template of the SRS

1. **Introduction**
   - 1.1 Purpose
   - 1.2 Document conventions
   - 1.3 Project scope
   - 1.4 References
2. **Overall description**
   - 2.1 Product perspective
   - 2.2 User classes and characteristics
   - 2.3 Operating environment
   - 2.4 Design and implementation constraints
   - 2.5 Assumptions and dependencies
3. **System features**
   - 3.x System feature X
     - 3.x.1 Description
     - 3.x.2 Functional requirements
4. **Data requirements**
   - 4.1 Logical data model
   - 4.2 Data dictionary
   - 4.3 Reports
   - 4.4 Data acquisition, integrity, retention, and disposal
5. **External interface requirements**
   - 5.1 User interfaces
   - 5.2 Software interfaces
   - 5.3 Hardware interfaces
   - 5.4 Communications interfaces
6. **Quality attributes**
   - 6.1 Usability
   - 6.2 Performance
   - 6.3 Security
   - 6.4 Safety
   - 6.x [others]
7. **Internationalization and localization requirements**
8. **Other requirements**

Appendix A: Glossary

Appendix B: Analysis models

The Structure of a Requirements Document

1. **Introduction**

The introduction presents an overview to help the reader understand how the SRS is organized and how to use it.

1.1 Purpose

Identify the product or application whose requirements are specified in this document, including the revision or release number.

1.2 Document conventions

Describe any standards or typographical conventions used, including the meaning of specific text styles, highlighting, or notations.

1.3 Project scope

Provide a short description of the software being specified and its purpose.

1.4 References

The Structure of a Requirements Document

2. Overall description

This section presents a high-level overview of the product and the environment.

2.1 Product perspective

Describe the product's context and origin.

2.2 User classes and characteristics

Identify the various user classes that you anticipate will use this product, and describe their pertinent characteristics.

2.3 Operating environment

Describe the environment in which the software will operate。

2.4 Design and implementation constraints

Describe any factors that will restrict the options available to the developers and the rationale for each constraint.

2.5 Assumptions and dependencies

An assumption is a statement that is believed to be true in the absence of proof or definitive knowledge.

The Structure of a Requirements Document

3. System features

Possible ways to arrange functional requirements: system feature, functional area, process flow, use case, mode of operation, user class, stimulus, and response

Example:

- 3.x System feature X

State the name of the feature in just a few words, such as “3.1 Spell Check.” Repeat section 3.x with its subsections 3.x.1 and 3.x.2 for each system feature.

- 3.x.1 Description

Provide a short description of the feature and indicate whether it is of high, medium, or low priority.

- 3.x.2 Functional requirements

Itemize the specific functional requirements associated with this feature.

The Structure of a Requirements Document

4. Data requirements

Use this section of the template to describe various aspects of the data that the system will consume as inputs, process in some fashion, or create as outputs.

4.1 Logical data model

A data model is a visual representation of the data objects and collections the system will process and the relationships between them.

Entity-relationship diagrams and UML class diagrams

4.2 Data dictionary

The data dictionary defines the composition of data structures and the meaning, data type, length, format, and allowed values for the data elements that make up those structures.

4.3 Reports

4.4 Data acquisition, integrity, retention, and disposal

The Structure of a Requirements Document

5. External interface requirements

Provide information to ensure that the system will communicate properly with users and with external hardware or software elements.

5.1 User interfaces

Describe the logical characteristics of each user interface that the system needs.

5.2 Software interfaces

Describe the connections between this product and other software components (identified by name and version), including other applications, databases, operating systems…

5.3 Hardware interfaces

Describe the characteristics of each interface between the software components and hardware components, if any, of the system.

5.4 Communications interfaces

The Structure of a Requirements Document

6. Quality attributes

This section specifies nonfunctional requirements other than constraints and external interface requirements.

6.1 Usability

6.2 Performance

6.3 Security

6.4 Safety

The Structure of a Requirements Document

7. Internationalization and localization requirements

Internationalization and localization requirements ensure that the product will be suitable for use in nations, cultures, and geographic locations.

8. [Other requirements]

Define any other requirements that are not covered elsewhere in the SRS.

Appendix A: Glossary

Define any specialized terms that a reader needs to know to understand the SRS, including acronyms and abbreviations.

Appendix B: Analysis models

This optional section includes or points to pertinent analysis models.

Example: Cafeteria Ordering System (COS)

1. Introduction

1.1 Purpose

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Cafeteria Ordering System (COS). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0.

1.2 Document Conventions

No special typographical conventions are used in this SRS.

1.3 Project Scope

The COS will permit Process Impact employees to order meals from the company cafeteria online to be delivered to specified campus locations. A detailed description is available in the *Cafeteria Ordering System Vision and Scope Document* [1], along with the features that are scheduled for full or partial implementation in this release.

1.4 References

1. Wiegers, Karl. *Cafeteria Ordering System Vision and Scope Document, www.processimpact.com/projects/COS/COS Vision and Scope.docx*
2. Beatty, Joy. *Process Impact Intranet Development Standard, Version 1.3, www.processimpact.com/corporate/standards/PI Intranet Development Standard.pdf*
3. Rath, Andrew. *Process Impact Internet Application User Interface Standard, Version 2.0, www.processimpact.com/corporate/standards/PI Internet UI Standard.pdf*

Example: Cafeteria Ordering System (COS)

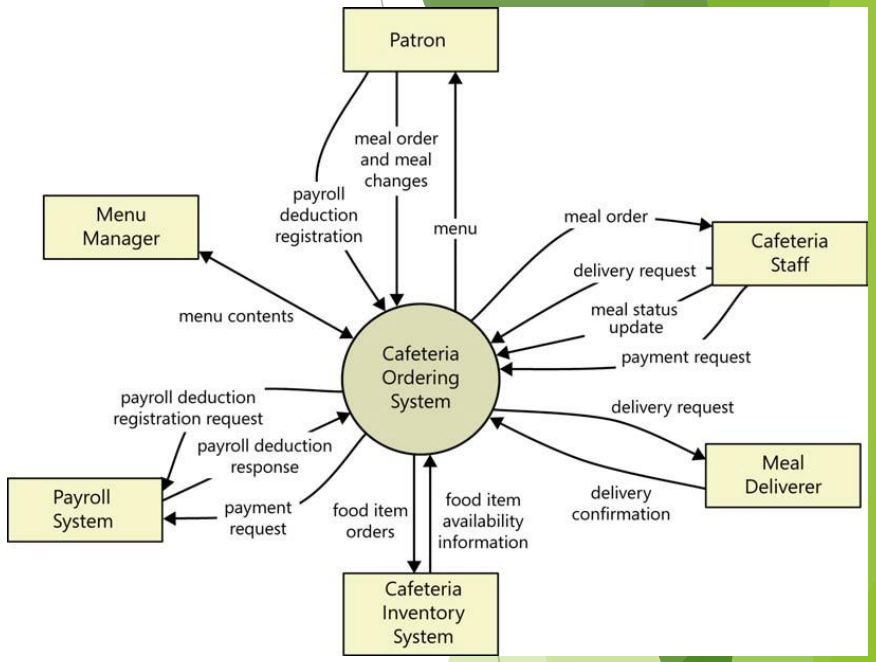
2. Overall Description

2.1 Product Perspective

The Cafeteria Ordering System is a new software system that replaces the current manual and telephone processes for ordering and picking up meals in the Process Impact cafeteria. The context diagram in Figure C-2 illustrates the external entities and system interfaces for release 1.0. The system is expected to evolve over several releases, ultimately connecting to the Internet ordering services for several local restaurants and to credit and debit card authorization services.

2.2 User Classes and Characteristics

User class	Description
Patron (favored)	A Patron is a Process Impact employee who wants to order meals to be delivered from the company cafeteria. There are about 600 potential Patrons, of which 300 are expected to use the COS an average of 5 times per week each. Patrons will sometimes order multiple meals for group events or guests. An estimated 60 percent of orders will be placed using the corporate intranet, with 40 percent of orders being placed from home or by smartphone or tablet apps.
Cafeteria Staff	The Process Impact cafeteria employs about 20 Cafeteria Staff who will receive orders from the COS, prepare meals, package them for delivery, and request delivery. Most of the Cafeteria Staff will need training in the use of the hardware and software for the COS.
Menu Manager	The Menu Manager is a cafeteria employee who establishes and maintains daily menus of the food items available from the cafeteria. Some menu items may not be available for delivery. The Menu Manager will also define the cafeteria's daily specials. The Menu Manager will need to edit existing menus periodically.
Meal Deliverer	As the Cafeteria Staff prepare orders for delivery, they will issue delivery requests to a Meal Deliverer's smartphone. The Meal Deliverer will pick up the food and deliver it to the Patron. A Meal Deliverer's other interactions with the COS will be to confirm that a meal was (or was not) delivered.



Example: Cafeteria Ordering System (COS)

2.3 Operating Environment

OE-1: The COS shall operate correctly with the following web browsers: Windows Internet Explorer versions 7, 8, and 9; Firefox versions 12 through 26; Google Chrome (all versions); and Apple Safari versions 4.0 through 8.0.

OE-2: The COS shall operate on a server running the current corporate-approved versions of Red Hat Linux and Apache HTTP Server.

OE-3: The COS shall permit user access from the corporate intranet; from a VPN Internet connection; and by Android, iOS, and Windows smartphones and tablets.

2.4 Design and Implementation Constraints

CO-1: The system's design, code, and maintenance documentation shall conform to the *Process Impact Intranet Development Standard, Version 1.3* [2].

CO-2: The system shall use the current corporate standard Oracle database engine.

CO-3: All HTML code shall conform to the HTML 5.0 standard.

2.5 Assumptions and Dependencies

AS-1: The cafeteria is open for breakfast, lunch, and supper every company business day in which employees are expected to be on site.

DE-1: The operation of the COS depends on changes being made in the Payroll System to accept payment requests for meals ordered with the COS.

DE-2: The operation of the COS depends on changes being made in the Cafeteria Inventory System to update the availability of food items as COS accepts meal orders.

Example: Cafeteria Ordering System (COS)

3. System Features

3.1 Order Meals from Cafeteria

3.1.1 Description

A cafeteria Patron whose identity has been verified can order meals either to be delivered to a specified company location or to be picked up in the cafeteria. A Patron can cancel or change a meal order if it has not yet been prepared. Priority = High.

3.1.2 Functional Requirements

Order.Place:	**Placing a meal order**
.Register:	The COS shall confirm that the Patron is registered for payroll deduction.
.No:	If the Patron is not registered for payroll deduction, the COS shall give the Patron options to register now and continue placing an order, to place an order for pickup in the cafeteria (but not for delivery), or to exit.
.Date:	The COS shall prompt the Patron for the meal date (see BR-8).
.Cutoff:	If the meal date is the current date and the current time is after the order cutoff time, the COS shall inform the Patron that it's too late to place an order for today. The Patron can either change the meal date or cancel the order.
Order.Deliver:	**Delivery or pickup**
.Select:	The Patron shall specify whether the order is to be picked up or delivered.
.Location:	If the order is to be delivered and there are still available delivery times for the meal date, the Patron shall provide a valid delivery location.
.Notimes:	The COS shall notify the Patron if there are no available delivery times for the meal date. The Patron shall either cancel the order or indicate that he will pick up the order in the cafeteria.
.Times:	The COS shall display the remaining available delivery times for the meal date. The COS shall allow the Patron to request one of the delivery times shown, to change the order to be picked up in the cafeteria, or to cancel the order.

3.2 Order Meals from Restaurants

[Details are not provided in this example. Quite a lot of the functionality described under 3.1 Order Meals from Cafeteria could likely be reused, so this section should just specify the additional functionality that addresses the restaurant interface.]

3.3 Create, View, Modify, and Delete Meal Subscriptions

[Details are not provided in this example.]

3.4 Create, View, Modify, and Delete Cafeteria Menus

[Details are not provided in this example.]

Example: Cafeteria Ordering System (COS)

4. Data Requirements

4.1 Logical Data Model

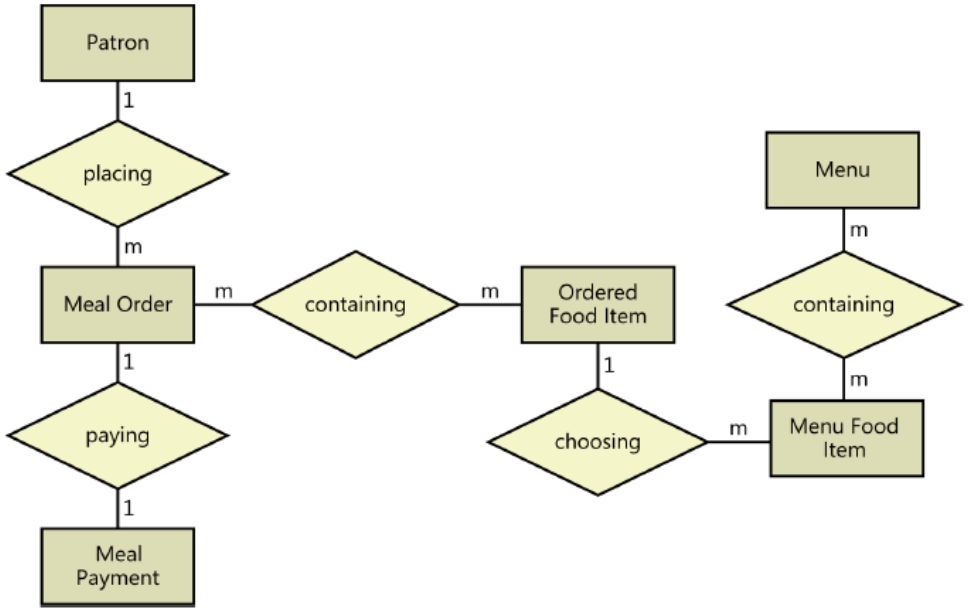


FIGURE C-3 Partial data model for release 1.0 of the Cafeteria Ordering System.

4.2 Data Dictionary

Data element	Description	Composition or data type	Length	Values
delivery instruction	where and to whom a meal is to be delivered, if it isn't being picked up in the cafeteria	patron name + patron phone number + meal date + delivery location + delivery time window		
delivery location	building and room to which an ordered meal is to be delivered	alphanumeric	50	hyphens and commas permitted

5. External Interface Requirements

5.1 User Interfaces

UI-1: The Cafeteria Ordering System screen displays shall conform to the *Process Impact Internet Application User Interface Standard, Version 2.0* [3].

UI-2: The system shall provide a help link from each displayed webpage to explain how to use that page.

UI-3: The webpages shall permit complete navigation and food item selection by using the keyboard alone, in addition to using mouse and keyboard combinations.

5.2 Software Interfaces

SI-1: Cafeteria Inventory System

SI-1.1: The COS shall transmit the quantities of food items ordered to the Cafeteria Inventory System through a programmatic interface.

SI-1.2: The COS shall poll the Cafeteria Inventory System to determine whether a requested food item is available.

SI-1.3: When the Cafeteria Inventory System notifies the COS that a specific food item is no longer available, the COS shall remove that food item from the menu for the current date.

SI-2: Payroll System

Example: Cafeteria Ordering System (COS)

6. Quality Attributes

6.1 Usability Requirements

USE-1: The COS shall allow a Patron to retrieve the previous meal ordered with a single interaction.

USE-2: 95% of new users shall be able to successfully order a meal without errors on their first try.

6.2 Performance Requirements

PER-1: The system shall accommodate a total of 400 users and a maximum of 100 concurrent users during the peak usage time window of 9:00 A.M. to 10:00 A.M. local time, with an estimated average session duration of 8 minutes.

PER-2: 95% of webpages generated by the COS shall download completely within 4 seconds from the time the user requests the page over a 20 Mbps or faster Internet connection.

PER-3: The system shall display confirmation messages to users within an average of 3 seconds and a maximum of 6 seconds after the user submits information to the system.

6.3 Security Requirements

SEC-1: All network transactions that involve financial information or personally identifiable information shall be encrypted per BR-33.

SEC-2: Users shall be required to log on to the COS for all operations except viewing a menu.

SEC-3: Only authorized Menu Managers shall be permitted to work with menus, per BR-24.

SEC-4: The system shall permit Patrons to view only orders that they placed.

Appendix A: Analysis Models

Figure C-4 is a state-transition diagram that shows the possible meal order statuses and the allowed changes in status.

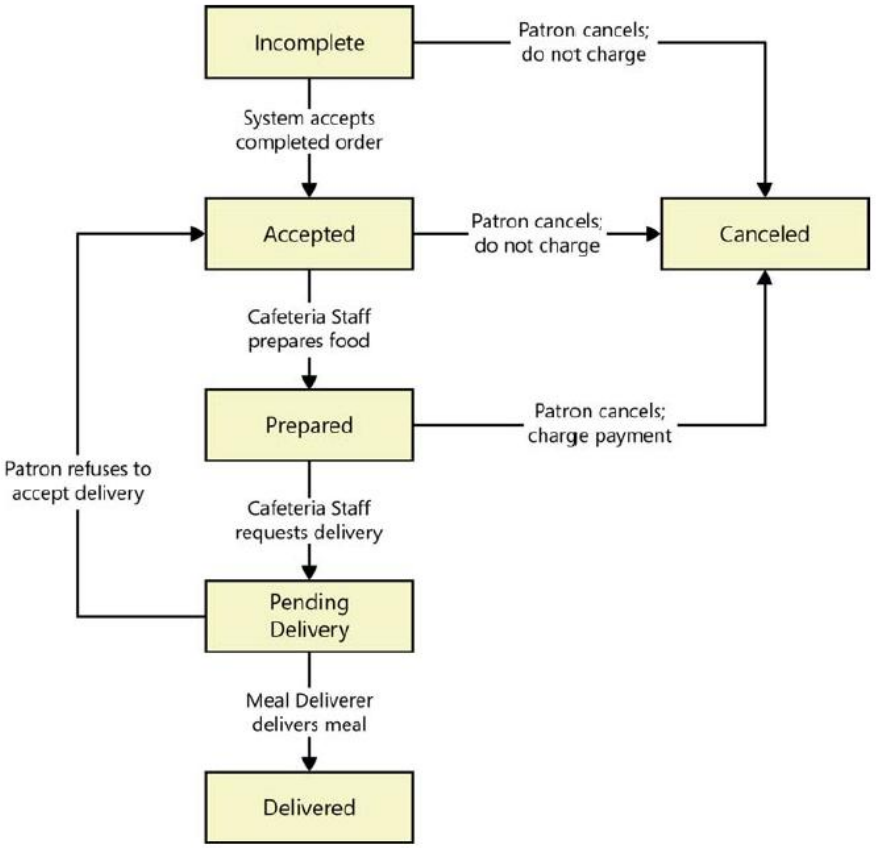


FIGURE C-4 State-transition diagram for meal order status.

Requirements document variability

- Information in requirements document depends on type of system and the approach to development used.
- Systems developed incrementally will, typically, have less detail in the requirements document.
- Requirements documents standards have been designed e.g. IEEE standard. These are mostly applicable to the requirements for large systems engineering projects.

In the next lecture…

- We will talk about requirements validation

Question session